

Rapport technique du POC de fine-tuning medical bilingue

1. Introduction

Ce document presente une synthese technique et critique du projet actuel de fine-tuning medical bilingue francais/anglais autour de Qwen3-1.7B. Le POC couvre quatre volets principaux :

- ? la preparation d'un dataset SFT et d'un dataset DPO a partir de sources medicales publiques ;
- ? l'entrainement d'un adaptateur LoRA en 4-bit sur la base unsloth/Qwen3-1.7B-unsloth-bnb-4bit ;
- ? une evaluation comparative entre le modele de base et un checkpoint SFT ;
- ? une trajectoire de deploiement via une API FastAPI connectee a un serveur vLLM.

Ce rapport ne se limite pas a presenter quelques metriques. Il cherche aussi a expliquer les choix methodologiques, a situer le niveau de maturite reel du systeme, a documenter les aspects cout/performance et a exposer clairement les limites actuelles du POC.

2. Perimetre et etat actuel du projet

Le depot correspond a un POC pedagogique deja bien avance sur les volets donnees, fine-tuning et evaluation, mais encore en construction sur la partie benchmarking d'inference et sur l'industrialisation du deploiement.

Les elements actuellement disponibles dans le projet sont les suivants :

- ? un pipeline notebook pour l'EDA, la normalisation, la deduplication et l'anonymisation des donnees ;
- ? un notebook d'entrainement SFT puis DPO avec Unsloth, PEFT et TRL ;
- ? un notebook d'evaluation comparative sur 500 exemples ;
- ? une API FastAPI minimale avec backend echo pour les tests locaux et backend vllm pour l'inference cible ;
- ? un script scripts/benchmark.py prevu pour mesurer la latence et le debit en inference ;
- ? une base de documentation sur le deploiement Google Cloud et l'usage de vLLM.

En revanche, les mesures consolidees de latence, de debit, de consommation GPU et les journaux complets de duree d'entrainement ne sont pas archives dans le workspace actuel. Il faut donc garder une distinction simple en tete : certaines conclusions s'appuient sur des artefacts d'evaluation bien presents, tandis que la partie cout/performance d'infrastructure reste pour l'instant davantage au niveau du cadrage et de la methode qu'au niveau d'un benchmark completement finalise.

3. Methodologie

3.1 Preparation des donnees

Le projet assemble des sources medicales heterogenes en francais et en anglais afin de couvrir a la fois des questions ouvertes et des QCM. Les sources mentionnees dans le depot sont notamment ANR-MALADES/MediQAI, nthngdy/frenchmedmcqa, keivalya/MedQuad-MedicalQnADataset et TsinghuaC3I/UltraMedical-Preference.

Pour un POC, le pipeline de preparation suit une logique claire et raisonnable :

1. chargement et inspection des jeux de donnees ;
2. normalisation vers un schema commun ;
3. construction des exemples SFT et DPO ;

4. deduplication des couples texte/reponse ;
5. echantillonnage par quotas ;
6. separation train/validation/test pour le SFT ;
7. anonymisation heuristique de certaines entites sensibles.

Cette approche presente trois avantages concrets. D'abord, elle permet de melanger des formats de supervision differents tout en conservant une interface commune. Ensuite, elle rend possible une evaluation distincte entre generation libre et QCM. Enfin, elle introduit un premier niveau de reduction du risque PII, meme si cette anonymisation reste encore insuffisante pour un contexte clinique reel.

3.2 Choix du modele de base

Le choix de Qwen3-1.7B ne vient pas d'une comparaison ouverte entre plusieurs modeles candidats : il etait impose par le cadre du projet. La vraie question technique est donc plus simple : est-ce un point de depart coherent pour un POC frugal, et reste-t-il defendable dans l'optique d'un passage a l'echelle sur une infrastructure raisonnable ?

Dans ce cadre contraint, les principaux arguments en faveur de ce socle restent les suivants :

- ? la taille 1.7B, suffisamment compacte pour rendre le fine-tuning et l'inference envisageables sur une infrastructure GPU modeste ;
- ? la disponibilite d'une variante unsloth-bnb-4bit, qui reduit fortement la pression memoire ;
- ? un bon alignement avec un usage bilingue instructionnel ;
- ? un cout de serving bien plus faible que celui d'un modele beaucoup plus grand, a performance potentiellement moins rentable dans le cadre du POC.

En pratique, meme si le choix initial etait impose, il reste compatible avec une logique d'iterabilite experimentale, de reproductibilite et de budget maitrise. Pour un POC qui cherche ensuite a monter en charge de facon pragmatique, ce n'est peut-etre pas le meilleur modele possible, mais c'est un point de depart tout a fait defensible.

3.3 Choix des techniques d'entrainement

Le projet combine deux techniques complementaires :

- ? LoRA pour le fine-tuning supervise ;
- ? DPO pour l'alignement par preferences.

Le recours a LoRA est bien adapte a ce contexte, a la fois pour des raisons de cout et de simplicite. En evitant de mettre a jour tous les poids du modele, LoRA reduit fortement la consommation memoire et le temps d'entrainement, tout en permettant une specialisation efficace sur le domaine medical. Le chargement en 4-bit pousse encore plus loin cette logique de frugalite.

Le recours a DPO se defend egalement bien. Une fois un premier alignement supervise obtenu, DPO permet de travailler la preference entre une bonne et une moins bonne reponse sans passer par un pipeline RLHF complet, beaucoup plus lourd a mettre en oeuvre. Pour un POC, c'est une facon assez naturelle d'explorer un second niveau d'alignement clinique tout en gardant une complexite operationnelle maitrisee.

3.4 Choix du deploiement et de vLLM

Le projet retient une architecture a deux niveaux :

- ? une API FastAPI qui porte le contrat applicatif ;
- ? un serveur vLLM pour l'inference du modele.

Ce choix tient bien la route sur le plan de la performance comme sur celui de l'architecture. vLLM est bien adapte au serving LLM : il apporte un serveur OpenAI-compatible, un meilleur debit que des appels de generation plus naifs, une gestion plus efficace du KV cache via PagedAttention et un batching continu utile en cas de concurrence. FastAPI reste de son cote une bonne couche d'integration pour porter le contrat HTTP, les routes /health et /generate, les logs et d'eventuelles regles metier.

Le backend echo conserve dans le depot ne doit donc pas etre interprete comme un backend de production. Il sert surtout de fallback tres leger pour la CI, les tests et le packaging Docker sans GPU.

4. Configuration experimentale

4.1 Jeux de donnees cibles

Le projet decrit deux datasets finaux :

- ? Maphe/medical-sft-5k pour l'entrainement supervise ;
- ? Maphe/medical-dpo-5k pour l'alignement par preferences.

Pour le SFT, la cible documentee est de 5 000 exemples d'entrainement, 500 exemples de validation et 500 exemples de test. L'evaluation disponible porte sur le split de test de 500 exemples, avec deux sous-ensembles :

- ? 267 exemples de texte libre ;
- ? 233 exemples de QCM.

4.2 Hyperparametres documentes

Le depot documente notamment les hyperparametres suivants.

Pour LoRA :

- ? $r = 16$
- ? `lora_alpha = 16`
- ? `lora_dropout = 0`
- ? `bias = none`
- ? modules cibles sur les projections d'attention et de MLP

Pour le SFT :

- ? 2 epochs
- ? batch size par device 32
- ? gradient accumulation 16
- ? learning rate $2e-4$
- ? optimiseur adamw_8bit
- ? sequence length 1024

Pour le DPO :

- ? 1 epoch
- ? batch size par device 4
- ? gradient accumulation 8
- ? learning rate $5e-5$
- ? $\beta = 0.1$

? optimiseur adamw_8bit

? sequence length 1024

Dans l'ensemble, cette configuration est cohérente avec un objectif de fine-tuning efficace sur matériel limité. Elle favorise l'experimentation rapide, même si elle ne garantit pas à elle seule la stabilité ni la meilleure qualité clinique possible.

5. Resultats observes

5.1 Resultats quantitatifs disponibles

L'artefact de comparaison le plus complet actuellement visible dans le workspace est le résumé notebooks/eval_results/qwen3_base_vs_sft_output_summary.json. Il compare le modèle de base au checkpoint SFT checkpoint-625 sur 500 exemples et intègre, pour le texte libre, METEOR, la similarité cosinus et la distance euclidienne.

Les principaux chiffres sont les suivants :

Metrique	Modele de base	Modele fine-tune SFT	Delta
METEOR moyen texte libre	0.1361	0.1653	+0.0292
Similarite cosinus moyenne texte libre	0.3762	0.3859	+0.0097
Distance euclidienne moyenne texte libre	0.2278	0.2231	-0.0047
Score QCM first-letter	0.0515	0.4378	+0.3863
QCM corrects	12	102	+90

Pour la distance euclidienne, une baisse est une amélioration.

Ce résumé fait aussi ressortir quelques tendances utiles :

- ? en texte libre, METEOR progresse nettement, la similarité cosinus n'augmente que légèrement en moyenne et la distance euclidienne baisse légèrement ;
- ? l'analyse par exemple reste toutefois partagée : sur 267 cas de texte libre, le fine-tune gagne 139 cas sur METEOR, seulement 113 sur la similarité cosinus contre 141 pour le modèle de base, mais 159 sur la distance euclidienne contre 108 pour le modèle de base ;
- ? en QCM, le gain est très net et constitue le signal positif le plus robuste du POC ;
- ? globalement, le rang agrégé reste favorable au checkpoint SFT, crédité de 368 rangs 1 contre 269 pour le modèle de base, avec 137 égalités.

5.2 Interpretation des resultats

L'amélioration la plus convaincante concerne les QCM. Le passage de 12/233 à 102/233 bonnes réponses montre que le fine-tuning a nettement renforcé la capacité du modèle à suivre un format de réponse contraint et à mieux sélectionner l'information attendue. C'est un résultat utile, car une grande partie des jeux de données médicaux structurent les connaissances sous forme de questions à choix multiple ou de réponses attendues très ciblées.

Sur le texte libre, la lecture demande davantage de nuance. Le gain de METEOR est réel, ce qui suggère une meilleure proximité lexicale moyenne avec les références. La distance euclidienne évolue elle aussi dans le bon sens, avec une baisse de 0.2278 à 0.2231, signe d'un rapprochement moyen de la distribution lexicale vers la référence. En revanche, la similarité cosinus reste plus réservée : la moyenne progresse légèrement, mais la médiane recule et le modèle de base reste meilleur sur davantage d'exemples individuels. Autrement dit, l'amélioration existe, mais elle n'est pas uniforme selon la métrique retenue.

Cette dissymétrie est assez cohérente avec un SFT qui apprend bien des patrons de réponse attendus, mais qui ne garantit pas encore une amélioration stable de la qualité clinique discursive. Le POC montre donc une progression réelle, surtout sur les formats structurés, sans permettre pour autant de conclure à une fiabilité suffisante pour un usage sensible ni à un gain sémantique uniforme sur la génération libre.

5.3 Analyse critique et nuancee

L'un des points positifs du projet est justement de ne pas s'arreter a un simple tableau de scores. Cela n'empêche pas de garder plusieurs limites importantes bien visibles.

Premiere limite : les metriques utilisees restent imparfaites pour juger une reponse medicale. METEOR, la similarite cosinus, la distance euclidienne ou le scoring par premiere lettre sont utiles pour une premiere comparaison, mais ils ne mesurent pas directement la securite, la factualite, la nuance clinique ou l'adequation a un contexte patient reel.

Deuxieme limite : l'evaluation disponible porte sur le checkpoint SFT, pas encore sur un benchmark final DPO consolide dans le depot. Il reste donc trop tot pour conclure serieusement sur l'apport effectif de DPO dans ce projet.

Troisieme limite : l'augmentation de la variance reste un signal d'alerte. Le modele fine-tune semble plus specialise, mais aussi moins regulier, y compris sur certaines metriques texte pourtant plus favorables en moyenne. Pour un POC medical, ce point compte beaucoup : quelques gains marquants ne compensent pas automatiquement plusieurs echecs critiques.

Quatrieme limite : aucun echantillon commente de reussites et d'echecs n'est encore archive dans le rapport. C'est un manque important, car une lecture clinique exige une analyse qualitative des erreurs, par exemple sur les omissions de red flags, les formulations trop affirmatives ou les reponses plausibles mais inexactes.

La conclusion la plus juste est donc la suivante : le POC montre un signal d'efficacite encourageant, surtout sur les taches structurees, mais il ne permet pas encore de revendiquer une qualite clinique robuste sur la generation libre.

6. Analyse cout/performance

6.1 Performance d'inference : latence et debit

Le projet contient deja un dispositif de mesure dans scripts/benchmark.py, prevu pour benchmarker la latence et le debit du backend vllm a travers l'API FastAPI. Le script collecte notamment :

- ? la latence par requete en millisecondes ;
- ? la latence moyenne, mediane et p95 ;
- ? le nombre de tokens generes ;
- ? le debit en tokens par seconde.

Ce choix de mesure est pertinent, car il observe la performance au niveau applicatif reel et pas seulement au niveau du moteur vLLM. Il integre donc le cout de la couche HTTP et donne une vision plus proche d'un service effectivement deploye.

En revanche, le depot actuel ne contient pas encore de sortie benchmark archivee. A ce stade, le rapport peut donc decrir proprement la methode de mesure et l'infrastructure cible, mais pas encore presenter des chiffres finaux de latence ou de debit valides experimentalement.

6.2 Cout d'infrastructure pour le deployment

La documentation GCP du projet recommande une VM n1-standard-4 avec 1 x NVIDIA T4, idealement en mode Spot. Le cout indicatif documente dans le depot est de l'ordre de 0,19 \$ a 0,26 \$ / heure pour cette configuration frugale.

Sur cette base, on peut etablir un cadrage simple du cout de serving GPU :

Duree d'utilisation GPU	Cout estime bas	Cout estime haut
1 heure	0,19 \$	0,26 \$
10 heures	1,90 \$	2,60 \$
30 heures	5,70 \$	7,80 \$

| 100 heures | 19,00 \$ | 26,00 \$ |

Il faut naturellement ajouter a cela le stockage persistant et, selon l'architecture retenue, le cout d'une petite instance CPU ou de quelques services reseau. Le projet rappelle d'ailleurs une bonne pratique tres simple : arreter la VM hors utilisation pour que la facturation du CPU et du GPU s'interrompe immediatement.

Cette strategie convient bien a un POC, avec un corollaire assez clair : tant que le service n'est pas automatise et benchmarke, le cout reste surtout pilote par le temps de machine allumee, et non encore par une capacite verifiee en requetes par seconde.

6.3 Cout d'infrastructure pour l'entrainement

Le besoin exprime porte aussi sur les heures GPU d'entrainement. Sur ce point, le depot documente la configuration d'entrainement et les artefacts produits, mais il n'archive pas dans le workspace actuel les journaux exploitables permettant de reconstituer de facon certaine :

- ? la duree SFT effective ;
- ? la duree DPO effective ;
- ? le nombre total d'heures GPU consommees ;
- ? le cout reel associe.

Il serait peu rigoureux d'inventer ces valeurs. Le plus propre est donc de distinguer deux niveaux :

- ? le cout reel observe, qui n'est pas encore consolide dans les fichiers disponibles ;
- ? le cout previsionnel, qui depend directement du nombre d'heures GPU effectivement consommees.

Avec la meme hypothese d'une machine GPU entre 0,19 \$ et 0,26 \$ par heure, le cout d'entrainement peut se lire de facon lineaire : cout entrainement = heures GPU x tarif horaire GPU.

Exemples de lecture :

Heures GPU cumulees	Cout estime bas	Cout estime haut
2 h	0,38 \$	0,52 \$
5 h	0,95 \$	1,30 \$
10 h	1,90 \$	2,60 \$
20 h	3,80 \$	5,20 \$

Ces ordres de grandeur montrent bien l'interet du couple Qwen3-1.7B + LoRA + 4-bit : le projet s'inscrit clairement dans une logique de fine-tuning et de serving a faible cout, surtout en comparaison de modeles plus grands ou d'un full fine-tuning.

6.4 Lecture cout/performance globale

Au stade actuel, le message technique le plus utile a retenir est le suivant :

- ? du cote qualite, le POC montre un gain tangible sur les taches structurees ;
- ? du cote cout, l'architecture choisie est volontairement frugale et compatible avec un budget etudiant ;
- ? du cote performance d'inference, la methode de benchmark existe deja mais les chiffres cibles restent a produire ;
- ? du cote entrainement, les choix LoRA et quantification 4-bit reduisent crediblement le cout, mais le total d'heures GPU doit encore etre consigne proprement.

En resume, la trajectoire cout/performance du projet parait saine, mais elle n'est pas encore completement demontree par des mesures d'exploitation archivees.

7. Comment passer a l'echelle vers un modele de classe Qwen 32B

Si l'objectif devient de dépasser un POC frugal pour viser une meilleure qualité de génération, une piste naturelle consiste à conserver l'architecture applicative actuelle tout en remplaçant le moteur de génération par un modèle beaucoup plus grand, par exemple un Qwen de classe 32B. L'intérêt d'un tel changement serait d'augmenter la capacité de raisonnement, la richesse des reformulations, la robustesse multilingue et la tenue des réponses longues. En contrepartie, le projet changerait de catégorie technique : on ne parlerait plus d'un serving léger sur GPU unique, mais d'un système d'inférence distribué et nettement plus coûteux.

L'idée la plus importante est donc la suivante : le passage à l'échelle ne suppose pas forcément de jeter l'architecture actuelle. La séparation déjà présente entre FastAPI et le backend d'inférence constitue au contraire une bonne base. L'API peut continuer à porter le contrat /health et /generate, la validation des entrées, les logs et d'éventuels garde-fous métier, tandis que la couche inférieure évolue d'un backend echo ou vllm simple vers un serving distribué capable de charger un modèle de très grande taille.

7.1 Ce qui change techniquement avec un modèle 32B

Le saut entre 1.7B et 32B n'est pas linéaire du point de vue opérationnel. Il change simultanément :

- ? la quantité de VRAM nécessaire pour charger les poids du modèle ;
- ? la bande passante mémoire requise pour maintenir une latence acceptable ;
- ? le nombre de GPU nécessaires pour l'inférence ;
- ? la complexité du déploiement réseau, du scheduling et de l'observabilité ;
- ? le coût unitaire par requête si aucune optimisation n'est mise en place.

Concrètement, un modèle de cette classe ne se déploie plus de façon réaliste sur une simple T4. Même en quantification agressive, il faut raisonner en termes de plusieurs GPU haut de gamme, avec parallélisme tensoriel et parfois pipeline parallelism selon le moteur retenu et la taille effective du contexte. Autrement dit, le changement de modèle implique un changement d'infrastructure bien avant d'impliquer un changement de code applicatif.

7.2 Architecture cible pour le serving à grande échelle

Dans une trajectoire de passage à l'échelle sérieuse, l'architecture la plus défendable resterait une architecture en couches :

Client -> FastAPI -> service d'inférence distribué -> cluster GPU -> modèle Qwen 32B

FastAPI conserverait plusieurs responsabilités utiles :

- ? stabiliser le contrat HTTP pour les clients ;
- ? appliquer des validations de payload et des bornes de sécurité ;
- ? journaliser les requêtes, les latences et les erreurs ;
- ? ajouter des politiques métier, par exemple sur les demandes urgentes ou hors périmètre ;
- ? masquer les détails d'implémentation du moteur de serving.

La couche d'inférence, en revanche, devrait monter en gamme. Dans cette hypothèse, vLLM reste un candidat crédible, mais il faudrait l'exploiter dans un mode distribué adapté aux grands modèles, avec :

- ? parallélisme tensoriel multi-GPU ;
- ? quantification si elle reste compatible avec la qualité attendue ;
- ? gestion stricte du KV cache et des longueurs de contexte ;
- ? supervision des temps de réponse, de la saturation GPU et des erreurs de génération.

L'API actuelle n'aurait alors pas besoin d'être refondue en profondeur. Le vrai travail porterait surtout sur le backend de serving, l'infrastructure GPU et l'observabilité de production.

7.3 Strategie de migration recommandee

Le point critique n'est pas seulement de choisir un plus gros modele, mais de le faire sans perdre la reproductibilite acquise sur le POC. Une trajectoire raisonnable pourrait se decomposer en quatre paliers.

1. conserver le contrat FastAPI actuel et remplacer d'abord le backend echo par un backend vllm reel sur un modele intermediaire ;
2. valider le benchmark de bout en bout sur une seule machine GPU plus solide que la T4 afin d'etablir une premiere base de latence, debit et VRAM ;
3. migrer ensuite vers un modele plus grand necessitant plusieurs GPU, avec parallelisme explicite et mesures de charge ;
4. seulement apres cette stabilisation, envisager un modele de classe 32B avec objectifs de SLA, budget et politiques de routage clairement definis.

Cette progression est importante, car elle evite de passer brutalement d'un POC peu couteux a une pile tres lourde sans zone intermediaire d'apprentissage. Dans beaucoup de projets, la bonne decision n'est pas de passer directement au plus gros modele possible, mais de verifier a partir de quel niveau de taille le gain qualitatif justifie reellement la hausse de complexite et de cout.

7.4 Impact sur le fine-tuning et l'alignement

Le passage a un Qwen 32B pose aussi une question strategique sur l'entrainement. Sur un petit modele, un fine-tuning LoRA en 4-bit reste compatible avec une logique de frugalite. Sur un modele beaucoup plus grand, meme un adaptateur LoRA devient plus exigeant en ressources, en stockage intermediaire, en temps de synchronisation et en orchestration. Le cout d'experimentation augmente donc fortement, meme si l'on evite toujours un full fine-tuning.

Dans ce contexte, deux options deviennent plus realistes qu'un simple portage du POC actuel :

- ? soit utiliser le grand modele principalement en inference, sans fine-tuning immediat, afin d'evaluer le gain qualitatif brut ;
- ? soit reserver le fine-tuning a des cas d'usage tres cibles, avec LoRA, jeu de donnees mieux nettoye, evaluation clinique plus stricte et infrastructure adaptee.

Cette distinction est importante pour la suite du projet. Si le besoin principal est d'ameliorer fortement la qualite de reponse, l'inference sur un grand modele peut suffire dans un premier temps. Si l'objectif est en plus de specialiser finement le comportement medical, alors il faut prevoir une vraie feuille de route MLOps pour les datasets, l'alignement, le versioning des adapters et les campagnes d'evaluation.

7.5 Arbitrage cout, performance et valeur metier

Le principal frein a un passage a l'echelle vers 32B ne sera probablement pas la faisabilite logicielle, mais l'economie du systeme. A ce niveau, le projet doit raisonner non plus seulement en heures GPU, mais en cout par requete utile, en latence acceptable pour l'utilisateur, en debit sous concurrence et en gain clinique reel par rapport a un modele plus petit.

Autrement dit, la bonne question n'est pas seulement : peut-on servir un Qwen 32B ? La bonne question est plutot : dans quelles conditions ce surcout se traduit-il par une amelioration suffisamment nette pour justifier l'infrastructure, les risques operationnels et l'effort de maintenance ?

Pour un projet medical, cette question est encore plus sensible. Un plus gros modele peut produire des reponses plus fluides et plus convaincantes, sans pour autant garantir a lui seul la surete clinique. Le passage a l'echelle doit donc etre pense comme un changement d'architecture globale : moteur plus puissant, certes, mais aussi evaluation plus exigeante, observabilite plus fine, garde-fous applicatifs plus stricts et gouvernance plus mature sur les usages autorises.

En ce sens, l'architecture actuelle du projet joue déjà un rôle utile : elle fournit un premier squelette séparant clairement la couche applicative de la couche d'inférence. Si le POC devait évoluer vers une offre plus ambitieuse, cette séparation permettrait justement de faire grandir le moteur de génération jusqu'à une classe 32B sans devoir redéfinir entièrement l'interface exposée aux utilisateurs.